SJ Project Planner Agent

Solution Architecture

Microsoft Azure · Microsoft Agent Framework · Foundry Hackathon

SJ Project Planner Agent is a **multi-agent AI system** that converts unstructured project conversations — meeting notes, emails, and chat threads — into structured, human-reviewed plan updates. Built entirely on **Microsoft Azure**, it follows the Microsoft Agent Framework **sequential-workflow pattern**. Every AI-proposed change must be approved by a human before it is committed to the official plan.

7	5+	39	1→5
AI Agents	Azure Services	Automated Tests	Auto-Scale Replicas

1 · COMPONENT ARCHITECTURE DIAGRAM

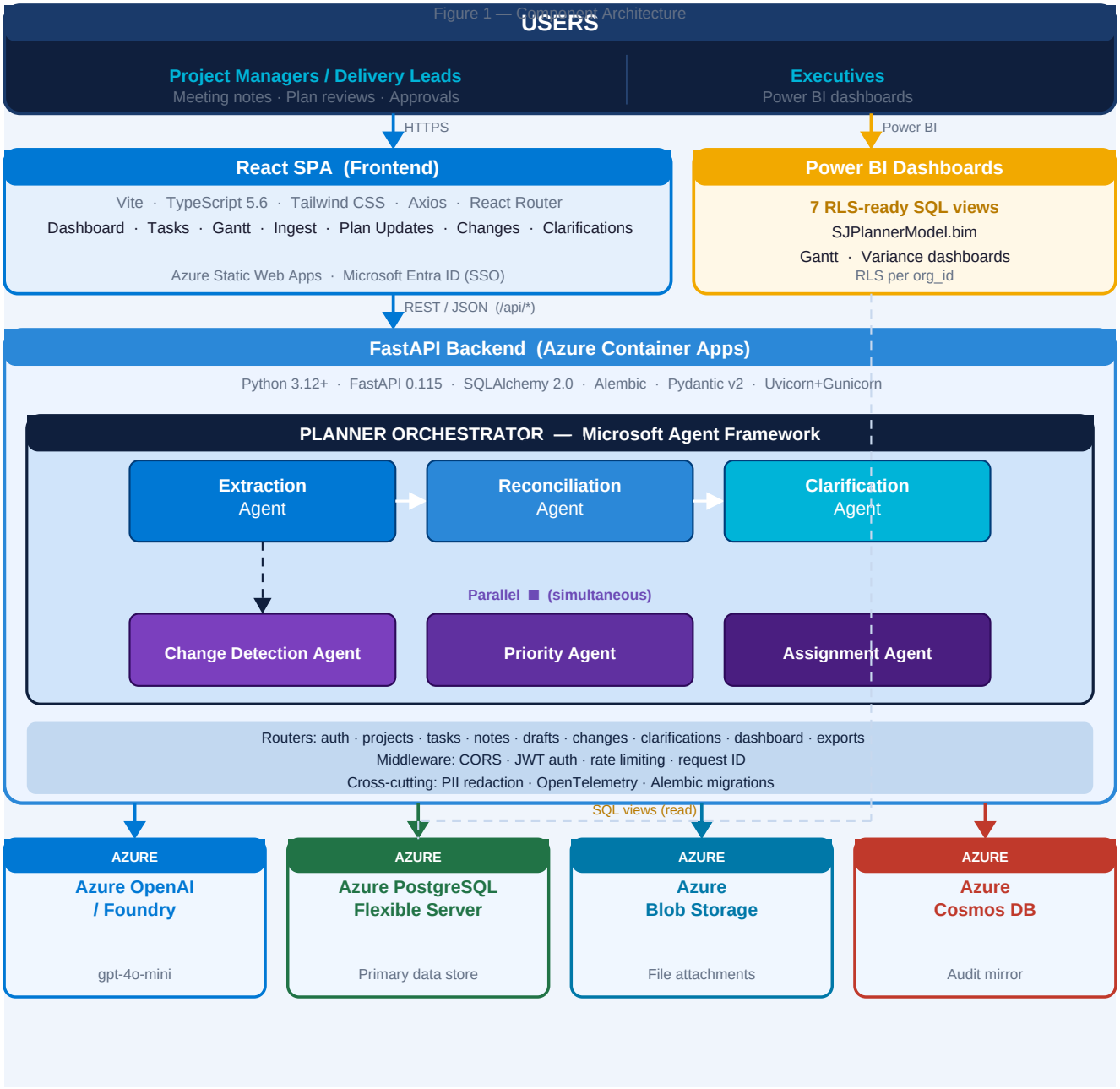

Figure 1 — Component Architecture

2 · THE 7-AGENT PIPELINE

All agents live in backend/app/agents/, coordinated by the Planner Orchestrator using the Microsoft Agent Framework sequential-workflow pattern.

1	**Extraction Agent** Sequential	Raw meeting note text	Calls Azure OpenAI (gpt-4o-mini) with a structured prompt. Extracts title, owner, due date, status, dependencies, confidence score, and a verbatim evidence quote. PII is redacted before the prompt leaves the backend.	*List of ExtractedItem objects*
2	**Reconciliation Agent** Sequential	Extracted items + live task list	Compares each item against the current plan using configurable similarity thresholds (HIGH=0.72, LOW=0.45). Labels each: create, update, or conflict.	*List of ReconciledItem objects*
3	**Clarification Agent** Sequential	Reconciled items	Identifies items where owner, due date, or confidence is missing or low. Auto-generates a specific, targeted question for each gap.	*List of Clarification questions*
4	**Change Detection Agent** Parallel	Live tasks + baseline snapshot	Diffs the current plan against the frozen baseline. Surfaces date shifts, owner reassignments, scope additions, status regressions.	*DiffReport with severity ratings*
5	**Priority Agent** Parallel	Live task list	Re-ranks tasks by composite score: urgency × dependency fan-in × status risk (blocked > in_progress > not_started).	*Ranked task list with scores & rationale*
6	**Assignment Agent** Parallel	Tasks without owners + team roster	Matches unowned tasks to team members using role/skill tags and current workload. Returns suggested owner with confidence score.	*List of Assignment suggestions*
7	**Planner Orchestrator** Orchestrator	All of the above	Sequences agents 1–3, runs 4–6 in parallel, persists the PlanUpdateDraft to PostgreSQL, fires draft.created webhook to Power Automate. Nothing written to the plan until human approves.	*PlanUpdateDraft record*

3 · END-TO-END DATA FLOW

1	User pastes meeting note on the Ingest page.
2	POST /api/projects/{id}/notes → note saved to PostgreSQL.
3	User clicks "Process" → POST .../notes/{id}/process.
4	Planner Orchestrator runs: Extraction → Reconciliation → Clarification (sequential) Change Detection + Priority + Assignment (parallel)
5	PlanUpdateDraft persisted to PostgreSQL (status = "pending"). Webhook fires → Power Automate → Teams adaptive card to task owner.
6	User reviews draft in UI: accept / reject / edit each change, answer clarification questions.
7	User clicks "Approve" → PATCH /api/drafts/{id}/approve → Tasks updated · ChangeLog written · Cosmos DB mirrored · Teams notification sent.

4 · API DESIGN

All endpoints require a JWT bearer token. Base path: /api/.

Resource	Endpoints	Notes
Auth	POST /auth/login · GET /auth/me	Local JWT or Microsoft Entra ID
Projects	GET/POST /projects · GET /projects/{id}	Multi-tenant by org_id
Tasks	GET/POST/PATCH/DELETE /projects/{id}/tasks	POST is AI-only by design
Notes	GET/POST /projects/{id}/notes · POST .../process	Entry point for AI pipeline
Drafts	GET /projects/{id}/drafts · PATCH /drafts/{id}/approve	Human review step
Changes	GET /projects/{id}/changes · GET .../diff	Change log + baseline diff
Clarif.	GET/PATCH /projects/{id}/clarifications	Open questions from AI
Dashboard	GET /projects/{id}/dashboard	KPI summary
Exports	GET /projects/{id}/export/csv · .../json	Bulk data export
Health	GET /api/health	Service status + feature flags

5 · TECHNOLOGIES USED

Frontend	
React 18	Component-based UI
TypeScript 5.6	Type safety
Vite 5.4	Build tool & dev server
Tailwind CSS 3.4	Utility-first styling
Axios 1.7	HTTP client with JWT interceptor
React Router 6.27	Client-side routing

Backend	
Python 3.12+	Primary language
FastAPI 0.115	Web framework · auto-generated OpenAPI docs
SQLAlchemy 2.0	ORM with async-ready sessions
Alembic 1.14	Database migration management
Pydantic v2 2.9	Request/response validation & settings
PyJWT 2.9	JWT generation & verification
Uvicorn 0.32	ASGI server (prod: Gunicorn workers)
Tenacity 9.0	Retry logic for external APIs
SlowAPI 0.1.9	Rate limiting per-endpoint, per-user
OpenTelemetry 1.27	Distributed tracing & metrics
pytest 8.3	39 automated tests

Azure Services	
Azure OpenAI / Foundry (gpt-4o-mini)	Powers Extraction, Reconciliation & Clarification agents
Azure Container Apps	FastAPI backend · auto-scale 1–5 replicas · managed identity
Azure PostgreSQL Flexible Server	Primary system of record: tasks, drafts, change log, users
Azure Blob Storage	File attachments (.eml, .docx, images) uploaded with notes
Azure Cosmos DB	Append-only event mirror for webhook audit replay
Azure AI Search	Hybrid vector + keyword retrieval over note corpus
Microsoft Entra ID	Production SSO · JWKS verification on every request
Azure Application Insights	OpenTelemetry traces, logs & metrics

Azure Key Vault	JWT_SECRET · OpenAI key · HMAC secret
Azure Static Web Apps	Compiled React frontend with global CDN

M365 & DevOps	
Power Automate	Teams adaptive cards · daily clarification reminders
Power BI	Gantt & variance dashboards · 7 SQL views · RLS per org_id
Bicep	Single IaC template — provisions all Azure in one command
GitHub Actions	CI: pytest 39 + TS + Vite · CD: Docker → GHCR → Bicep → Container App
Azure OIDC	Passwordless deployment — no long-lived secrets in CI
Docker	Backend image with Alembic migrations in entrypoint

6 · SCALABILITY

Compute (Container Apps)	Auto-scales 1→5 replicas · stateless FastAPI · any replica serves any request · scale limit configurable in Bicep
Database (PostgreSQL)	Vertical scaling zero downtime · read replicas for Power BI · pgBouncer pooling · multi-tenant org_id on every table
AI / LLM (Azure OpenAI)	TPM quota adjustable per deployment · multiple model deployments · deterministic offline fallback in all 7 agents
Storage	Blob Storage auto-scales to petabytes · Cosmos DB Serverless tier scales with requests, zero idle cost
Multi-Tenancy	Every entity scoped by org_id · JWT carries org_id claim · all endpoints enforce isolation · new org = 1 DB row
Search at Scale	Azure AI Search wired for hybrid retrieval — switch without changing agent interfaces
Observability	OTel traces to App Insights · alerts on latency/errors/token usage · Application Map · KQL log queries

7 · SECURITY ARCHITECTURE

Concern	How it is addressed
Authentication	JWT tokens (local dev) or Microsoft Entra ID JWKS verification (production)
Authorisation	RBAC: admin · reviewer · viewer — enforced via FastAPI Depends() on every endpoint
Tenant isolation	Every query filtered by org_id; cross-tenant access impossible without valid token for that org
Secret management	All secrets in Azure Key Vault; referenced as secretref: — never plaintext in env vars or source
PII protection	PiiRedactionService strips emails, phone numbers & names before any text reaches the LLM
Transport security	All external traffic HTTPS; internal Azure calls use mTLS managed identity
Rate limiting	SlowAPI enforces per-endpoint, per-IP limits (e.g. 30 req/min on note processing)
Supply chain	GitHub Actions CI runs dependency audits; Docker base image is pinned
Webhook integrity	Power Automate webhooks signed with HMAC-SHA256; signatures verified on receipt